# WealthForge Pro- Advanced Portfolio Management Platform

Frontend (Angular)
↓
API Gateway
↓
Eureka Service Discovery
↓
Multiple Spring Boot Microservices
↓
Databases (MySQL / MongoDB / Redis)
↓
RabbitMQ (Messaging)

## Backend (Eclipse + Spring Boot)

You will create **separate Spring Boot projects** for each microservice.

For each service you need:

**Required Dependencies:**

- Spring Web
- Spring Data JPA
- MySQL Driver
- Spring Security
- JWT Library
- Eureka Client
- Lombok
- Validation
- Swagger

For Gateway:

- Spring Cloud Gateway
- Eureka Client

For Config:

- Spring Cloud Config Server

This is a **Microservices Architecture** (not monolithic).

How Many Microservices & Their Names

You should create **8 Microservices**

**1⃣ Auth Service**

Handles:

- Register
- Login
- Logout
- OTP verification
- JWT generation

Database: MySQL

**2⃣ Investor Service**

Handles:

- Investor profile
- List advisors
- Search advisor
- View assigned advisor

Database: MySQL

**3⃣ Advisor Service**

Handles:

- Advisor profile
- List assigned investors
- Give advice

- View advised investors

Database: MySQL

**4⃣ Portfolio Service**

Handles:

- Portfolio creation
- Portfolio value
- Performance tracking
- Fund transfer

Database: MySQL

**5⃣ Trading Service**

Handles:

- Buy stocks
- Sell stocks
- Order execution
- Transaction history

Database: MySQL

**6⃣ Market Data Service**

Handles:

- Real-time stock price
- ETF data
- Market updates

Database:

- Redis (for caching)
- External API integration

**7⃣ Notification Service**

Handles:

- Real-time notifications
- Order success

- Advice notifications

Technology:

- WebSocket
- RabbitMQ

**8⃣ Admin Service**

Handles:

- Add company
- Delete company
- Update company
- View all investors
- View all companies

Database: MySQL

## 🔐 AUTH SERVICE – What To Create

**Entities:**

- User
- Role
- OTPVerification

**APIs:**

- /register
- /login
- /verify-otp
- /logout

**Security:**

- Spring Security
- JWT token
- Role based authorization

Flow:

1. User registers
2. OTP sent
3. Verify OTP
4. Login
5. JWT generated
6. JWT sent in header

**FRONTEND (Angular)**

Tools:

- Angular CLI
- Angular Material
- RxJS
- Bootstrap or CSS
- ngx-toastr (notifications)

**Angular Modules to Create**

**Authentication Module**

- Login Component
- Register Component
- OTP Verification Component

**Investor Dashboard**

- Portfolio View
- Buy Stock
- Sell Stock
- Transfer Funds
- Transaction History

**Advisor Dashboard**

- List Investors
- Advice Form
- Advice History

**Admin Dashboard**

- Companies Table
- Add/Edit/Delete Company
- Investor List

**Services in Angular:**

- auth.service.ts
- investor.service.ts
- advisor.service.ts
- portfolio.service.ts
- trading.service.ts
- admin.service.ts

## Database Design

Use MySQL.

**Auth Database Tables**

**users**

- id
- name
- email
- password
- role
- mfa_enabled
- verified

**otp_verification**

- id

- user_id
- otp_code
- expiry_time

**Investor Database**

**investors**

- id
- user_id
- advisor_id
- balance

**Portfolio Database**

**portfolio**

- id
- investor_id
- total_value

**portfolio_assets**

- id
- portfolio_id
- asset_type
- quantity
- buy_price

**Trading Database**

**transactions**

- id
- investor_id
- asset_symbol
- type (BUY/SELL)
- quantity
- price

- timestamp

**Admin Database**

**companies**

- id
- company_name
- symbol
- sector
- current_price

**Admin Database**

**companies**

- id
- company_name
- symbol
- sector
- current_price

## ROLES & PERMISSIONS

👤 Investor

Can:
✓ Register
✓ Login
✓ Buy
✓ Sell
✓ Transfer
✓ View history
✓ Search advisor

Cannot:
✕ Add companies
✕ View all investors
✕ Access admin APIs

👨💼 **Financial Advisor**

Can:
✓ Register
✓ Login
✓ List assigned investors
✓ Give advice
✓ View advised investors

Cannot:
✕ Trade stocks
✕ Add companies

**👨‍💻 Admin**

Can:
✓ Add company
✓ Delete company
✓ Update company
✓ View all investors
✓ View companies

Cannot:
✕ Buy/Sell
✕ Access investor private portfolio

## Security Implementation

You must implement:

**1⃣ JWT Authentication**

- After login generate JWT
- Send in header:

Authorization: Bearer <token>

**2⃣ Role Based Authorization**

Use:

@PreAuthorize("hasRole('ADMIN')")

Roles:

- ROLE_INVESTOR
- ROLE_ADVISOR
- ROLE_ADMIN

### 3 OTP (MFA)

Flow:

1. Register
2. Generate 6-digit OTP
3. Store in DB
4. Send via email
5. Verify before login

## 4 Password Encryption

Use: BCryptPasswordEncoder

## 8 CI/CD & Deployment

Use:

**Docker**

Each microservice → Dockerfile

**Docker Compose**

Run:

- MySQL
- Redis
- RabbitMQ
- All microservices

**Jenkins**

- Build project
- Run tests
- Create Docker image
- Deploy

# Final Architecture Summary

Frontend (Angular)
⬇
API Gateway
⬇
Eureka
⬇
Auth Service
Investor Service
Advisor Service
Portfolio Service
Trading Service
Market Data Service
Admin Service
Notification Service
⬇
MySQL + Redis + RabbitMQ